

Control laws in the Rattlesnake MIMO random environment

Reference for the six control laws and one diagnostic in `control_laws/`, as the code stands 2026-09-20. Written against the source, with line numbers, for someone who has to modify it.

This document describes what each law computes, in what order, with which safeguards, and what it does *not* control. Where a law's own docstring disagrees with its code, the code is described and the disagreement is flagged. Several sections record measured failures and their corrections, because in every case the failure is the clearest available explanation of why the code is shaped the way it is.

0. Common interface and notation

Rattlesnake offers three calling conventions. A **plain function** is called afresh every control cycle and holds no state. A **class** (`control_python_function_type = 2`) is constructed once, receives `system_id_update()` after identification, and `control()` every cycle. A **generator** is primed with `next()` and driven with `send()`.

A loader detail that will cost you an afternoon Rattlesnake recognises a class as a control law only if `system_id_update` and `control` are in the class's own `__dict__`, not merely inherited. That is why `optimal_diagonal_control_fast` carries two methods whose entire body is a `super()` call: without them the class is invisible in the “Control Python Function” dropdown despite fully implementing both. Relatedly, control-law files are loaded standalone via `importlib.spec_from_file_location` with no package context, so a relative import fails outright — which is why the files that need a sibling module load it by explicit path rather than with `from .control_laws import ....`

Dimensions and symbols

On this article: **N = 6 drives**, **M = 8 control channels**, **F = 2049 FFT lines, of which 901 are in band** (100–1000 Hz). Every law is written shape-agnostically and takes these from its inputs at runtime.

H	the FRF, $(F, M, N) = (2049, 8, 6)$ complex. One bin's H is 8×6 .
G_{xx}	the specification CPSD, (F, M, M) , Hermitian per line.
X	the commanded drive CPSD, (F, N, N) . This is what every law returns.
$Y = H X H^H$	the predicted response CPSD, (F, M, M) .
y	the target diagonal, $\text{real}(\text{diag}(G_{xx}))$, shape (2049, 8).

The reachability problem, stated once

With 8 control channels and 6 drives, $H H^+ \neq I_6$. The set of achievable response diagonals is the image of the PSD cone under $X \rightarrow \text{diag}(H X H^H)$, and when $M > N$ that is a *proper* subset of the positive orthant. A perfectly reasonable flat specification can be partly unreachable at some lines and not others. No control law gets outside that set, and a law that integrates toward a target outside it climbs until something stops it. Two channels on this rig — 13X+ and 14X+, reported as channels 7 and 8 — sit far enough outside that every law in this document gives them up; they read 10–13 dB rms in every run. §9 covers the diagnostic that quantifies this.

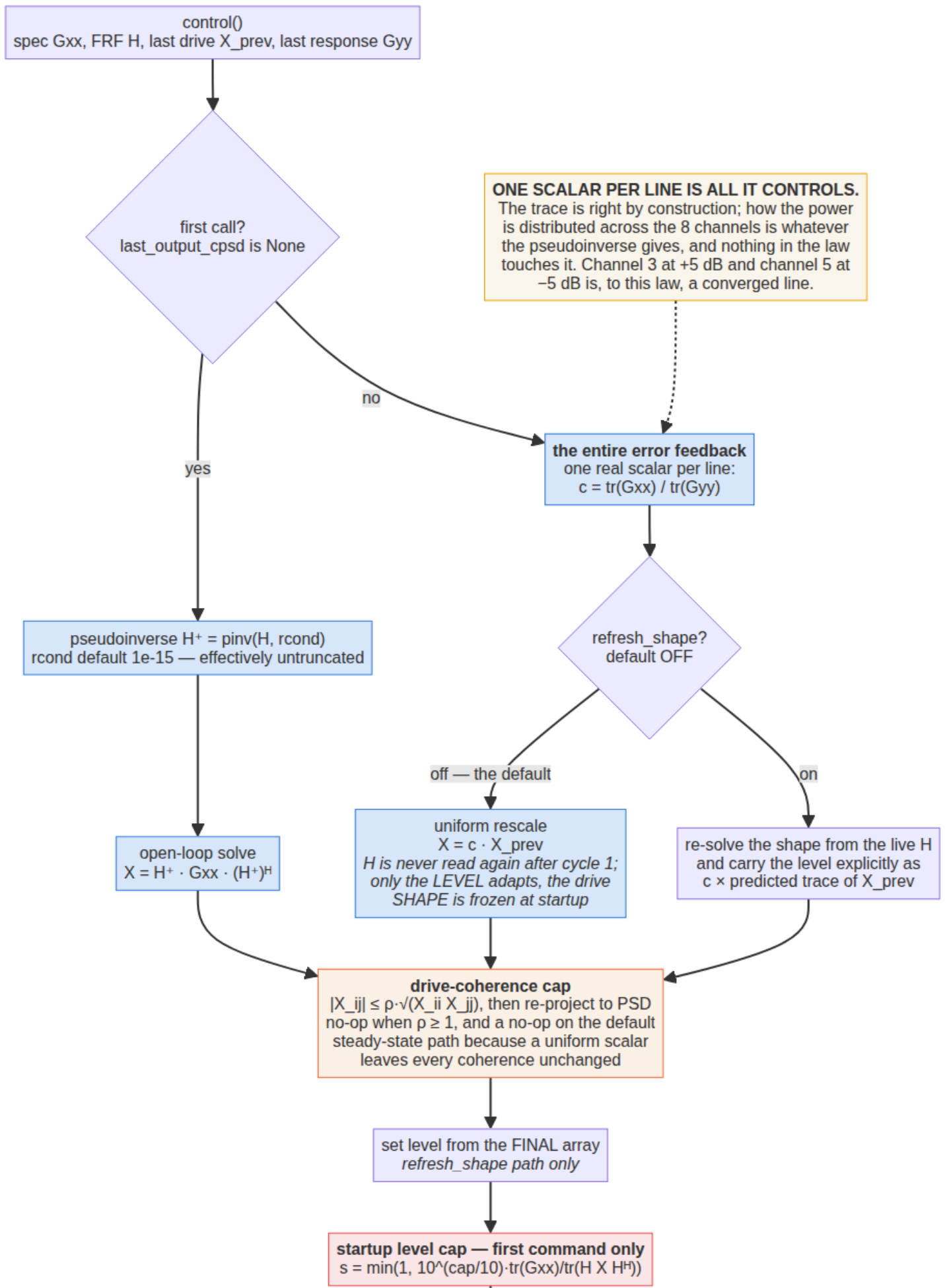
Three shared safeguards

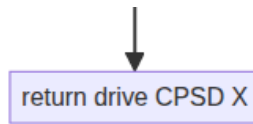
Most of the laws share the same three protective mechanisms, described in full in §7 and §8 rather than repeated per law:

- **The drive-coherence cap** bounds pairwise drive-to-drive coherence. It works two different ways: the optimal-diagonal family solves *subject to* it as an SDP constraint, while every other law post-processes it onto an already-computed solve. That distinction, not the loop type, is what predicts whether the cap costs anything.
- **The startup level cap** clamps the first command only.
- **Two live-FRF guards** protect the optimal-diagonal family against a diverging live FRF estimate.

1. `match_trace_pseudoinverse` — function, `control_laws.py:520`

One scalar per frequency line. The simplest law here, and the one whose limitation is easiest to state exactly.





`match_trace_pseudoinverse`. The two branches, and the ordering that took two attempts to get right.

The algorithm

The law branches on whether `last_output_cpsd` is `None` — that single test is all that distinguishes startup from steady state.

Cycle 1 (line 638–640). $H^+ = \text{pinv}(H, \text{rcond})$ is 6×8 ; $X = H^+ G_{xx} (H^+)^H$ is 6×6 . Because this is a congruence, X is PSD automatically whenever the specification is. The predicted response is $(H H^+) G_{xx} (H H^+)^H$ — the specification projected onto the at-most-6-dimensional column space of H . The residual is discarded, and the law neither measures nor reports it.

Cycle 2 onward (line 664, 676). $c = \text{tr}(G_{xx}) / \text{tr}(G_{yy})$, one complex-valued scalar per line, and $X \leftarrow c \cdot X_{\text{prev}}$. Both traces are sums over the eight channel autospectra, so this is a power ratio. It is a pure unity-gain integrator on level: no proportional term, no rate limit, no step bound. With `refresh_shape` off — the default — **H is never read again after cycle 1**; only the level adapts while the drive shape stays frozen at whatever the startup solve produced.

Ordering (lines 688, 691, 695). Coherence cap, then the explicit level set on the refresh path, then the startup cap. The cap is applied unconditionally even though on the default steady-state path it is a no-op — a uniform positive scalar multiplies $|X_{ij}|$ and $\sqrt{X_{ii}X_{jj}}$ equally, so coherence is unchanged — so that a mid-test law switch cannot arrive with an uncapped drive and bypass it.

Why the level is set from the final array On the `refresh_shape` path the level cannot simply be inherited through the coherence cap, because `_cap_drive_coherence` is *not idempotent*: its PSD re-projection moves the level by a bin-conditioning-dependent amount, and inheriting through it compounds the shift every cycle. Measured on run 01's identification with the cap at 0.95, bin 134 went +0.33 dB at cycle 5 and +28.44 dB at cycle 6, dragging a law that had been at +0.0001 dB to +2.49 dB aggregate. Setting the level from the final array also makes the constant-FRF case exact: with H unchanged the refresh path reproduces the default path bit for bit.

What it does not control

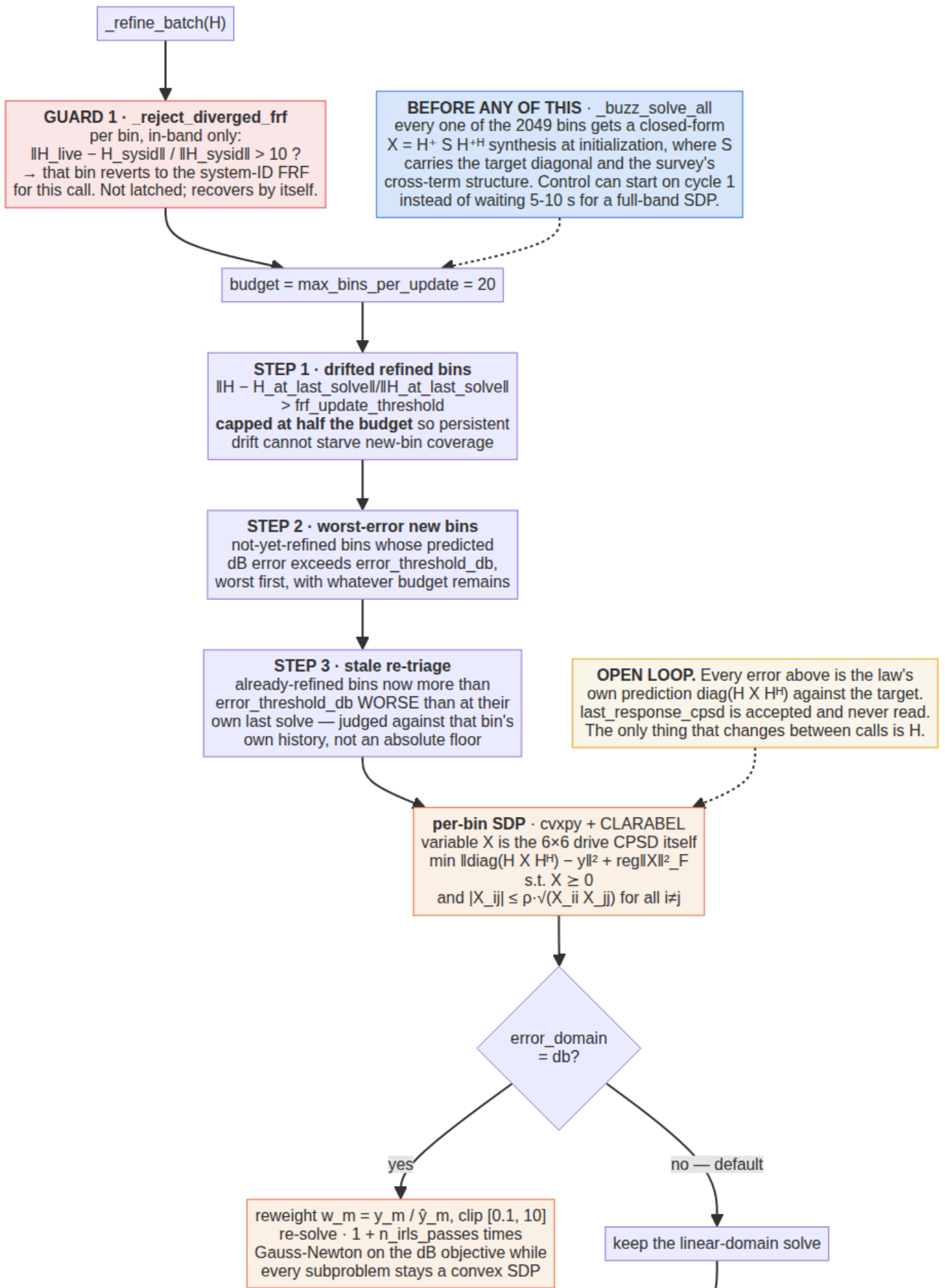
The feedback quantity is the *sum* over all eight channels. A line can be perfectly on-trace while individual channels are arbitrarily far off — channel 3 at +5 dB and channel 5 at −5 dB is, to this law, converged. It does not control per-channel level, response cross terms, response coherence or response phase. It has no absolute level ceiling after cycle 1, and cannot be given one: position 3 of its parameter string is deliberately skipped so that strings are not silently portable between law families.

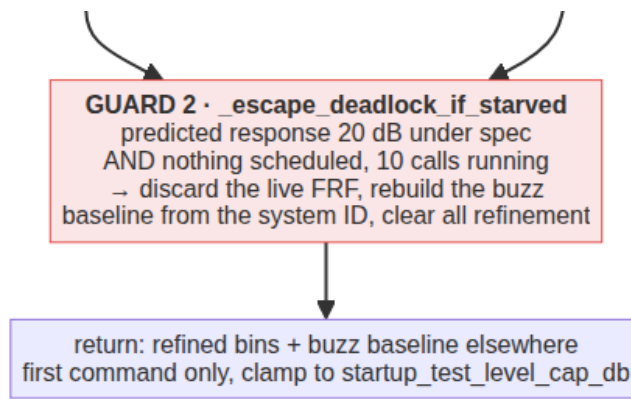
Arguments it accepts and never reads: `warning_levels`, `abort_levels`, both noise CPSDs, both sysid CPSDs, `multiple_coherence`, `frames`, `total_frames`. Lines with poor input/output coherence are not de-weighted or gated in any way.

Documentation gap The law's own docstring (lines 604–609) documents three parameter positions. The parser reads five. `refresh_shape` at position 4 is live and undocumented there; a value passed at position 3 is silently ignored.

2. optimal_diagonal_control — class, optimal_diagonal_control.py:167

A per-bin semidefinite program, spent under a budget. The most accurate law here, and the one with the most machinery.





optimal_diagonal_control: the buzz baseline, the three-step budgeted scheduler, the per-bin SDP, and the two guards.

The formulation

Per frequency line, with X the 6×6 drive CPSD itself as the decision variable:

```

minimize_X || diag(H X H^H) - y ||^2 + reg * ||X||_F^2
subject to X >= 0 (the PSD cone)
           |X_ij| <= max_drive_coherence * sqrt(X_ii * X_jj) for all i != j
  
```

Both constraints are convex — the coherence one is $|affine| \leq concave$ — so this is a genuine SDP with no manifold optimisation. It is built once per (M, N) shape and reused across all 2049 bins through `cvxpy Parameters`, rather than paying DCP canonicalisation per solve. Solver is CLARABEL.

Why H cannot be a Parameter $\text{diag}(H X H^H)$ is bilinear in H — H appears on both sides of X — which is not affine-in-parameter and so not DPP. Each channel's diagonal term is instead reformulated as a linear functional of X : $\text{diag}(H X H^H)[m] = \text{real}(\text{sum}(W_m \cdot X))$ with $W_m = \text{outer}(h_m, \text{conj}(h_m))$, and W_m is computed in plain numpy per bin at negligible cost.

The buzz baseline — why every bin starts somewhere sensible

A full SDP costs roughly 20–30 ms per bin, so solving the whole band up front would stall the first command by five to ten seconds. Instead every one of the 2049 bins gets a closed-form $X = H^+ S H^{+H}$ synthesis at initialisation, where S carries the target diagonal and the survey's measured cross-term structure (or zero cross terms if no survey CPSD was supplied). Control can start on cycle 1, and the SDP budget is then spent only where the baseline is measurably bad. The baseline is deliberately left uncapped, because it draws its cross terms from an independent-drive survey and so has no dependent-drive problem to begin with.

The three-step scheduler

Each call spends at most `max_bins_per_update` solves — 20 by default — across three steps in order:

1. **Drifted refined bins.** Already-refined bins whose H has moved more than `frf_update_threshold` in relative Frobenius norm *since that bin's own last solve*. **Capped at half the budget**, so that persistent or noise-driven drift cannot starve new-bin coverage. Note this step takes bins in ascending frequency order, not worst-drift first — unlike steps 2 and 3, which sort by error.
2. **Worst-error new bins.** Not-yet-refined bins whose predicted dB error exceeds `error_threshold_db`, worst first, with whatever budget remains. A bin already under the threshold is never marked refined and is re-scanned every call, so it gets picked up later if H moves and its error rises.
3. **Stale re-triage.** Already-refined bins now more than `error_threshold_db` *worse than they were at their own last solve*. Judged against that bin's own history rather than an absolute floor, because many bins — near a structural null, for instance — can never get under an absolute threshold no matter how often they are re-solved, and comparing to one would burn the whole budget forever on bins that were never going to improve.

Step 3 exists to close a specific gap: step 1's drift test is a proxy (has H moved a lot?) for the real question (is this bin's accuracy bad now?), and once every bin has been refined, step 2 can never reconsider one again. Step 3 only ever spends budget the other two did not use, so it changes nothing about a fresh start's convergence.

The dB objective, and why it is reached by IRLS here

Minimising $\sum(\hat{y}_m - y_m)^2$ in *linear* units caps what abandoning a channel can cost at that channel's own target squared — drive its response to zero and the penalty stops growing. The solver therefore sells the hardest channels to buy the easy ones, and is correct to, given what it was asked to minimise. On this frame, with row gains spanning 3.8 to 10.3 dB, that cost channels 13X+ and 14X+ about 5 dB each.

In dB the penalty has no such ceiling. But the dB objective is **not convex**, so it cannot be the SDP's objective. This class reaches it by iteratively reweighted least squares instead: weight each residual by $w_m = y_m/\hat{y}_m$, which is the Gauss-Newton step for least squares on $\log(\hat{y}/y)$, and iterate solve $\rightarrow$ reweight $\rightarrow$ solve. Every subproblem stays a convex SDP, so the coherence-cap constraint and the whole demote-to-the-safe-path design stay intact. Weights are normalised by y_m so that $w = 1$ exactly when a channel is on target — which keeps reg meaning what it meant before — and clipped to $[0.1, 10]$, a 20 dB span either way, so that a channel the current iterate has abandoned by orders of magnitude cannot make the next subproblem about nothing else.

The weighting had to go in through the existing parameters Setting $W_m \leftarrow w_m \cdot \text{outer}(h_m, \text{conj}(h_m))$ and $y \leftarrow w \cdot y$ gives $\sum w_m^2 (\hat{y}_m - y_m)^2$ exactly. A separate weight Parameter multiplying that expression is parameter-times-parameter, which cvxpy rejects as non-DPP — tried 2026-09-17.

What kind of loop this is

This law has been mislabelled twice, in opposite directions. Reports before 2026-09-17 called it a feedback law. The correction made then called it open loop, and that over-shot: it understates the law just as badly, because it reads as though the plant model were frozen, when re-solving against a moving FRF is the whole of the 2026-09-19 work. Two separate loops are at issue and the label has to name both.

Closed on the identified plant model. Both `control()` and `system_id_update()` hand the *live* transfer function to `_refine_batch` every cycle. Step 1 re-solves bins whose H has moved more than `frf_update_threshold` since that bin's own last solve. Step 3 re-solves bins whose predicted error has degraded, recomputed against the *current* H , worst-first. Step 3 is the one that matters for a plant whose FRF changes with level: the drift can sit well under the gate and still be caught, because the error is always re-evaluated with the new H rather than compared to the gate. This is what runs 29, 31 and 32 demonstrated — 17.03, 18.44 and 16.22 V² with the FRF update on, against 153, 485 and 245 V² with it off.

Open on the response error — until 2026-09-21, unconditionally; now only while `response_trim_gain` is 0, which is the default. The solve minimises the *predicted* response $\text{diag}(H X H^H)$ against the target. With the trim off, `last_response_cpsd` is accepted and never read, and `last_output_cpsd` is read in exactly one place — as a boolean “is this the first command” flag for the startup cap. Any error the FRF does not explain is therefore invisible: bias in the $H1$ estimate, extraneous input, drive clipping, a nonlinearity that never appears in a linear FRF. There is no integral action, so such an offset simply stands.

The accurate one-line description is an **indirect adaptive (self-tuning) regulator**: fast model update, optional slow error trim. The self-consistency diagnostic printed each cycle is the instrument for telling the two halves apart — if the law's own predicted error stays near 0 dB while Rattlesnake's measured Response Error panel stays high, the gap is model mismatch, not a bug in the scheduling or the SDP, which would be self-consistent by construction.

The response-error trim — closing the second loop

`_update_response_trim` carries a per-bin, per-channel multiplicative correction t on the solver's target, integrating the measured error. **Off by default** (`response_trim_gain = 0`), in keeping with every other behaviour change in this file: with the gain at zero the trim array is never allocated, `_eff_target` returns the specification slice itself, and the command is bit-identical to the code before the trim went in — verified on real run-29 data, $\max |new - committed| = 0.000e+00$.

Which error signal, and why it is not the obvious one. The tempting choice is measured-versus-predicted, $10\log_{10}(z/p)$, since that is literally the model mismatch. It does not work as an integrator input. For a constant multiplicative plant bias b it reads $10\log_{10}(b)$ for ever, whatever the trim does, because shrinking the target shrinks prediction and measurement together. It is a mismatch *readout*, not an error: driven by it, the integrator runs to its clamp and stops, and the clamp happening to sit at the right value would be a coincidence rather than a design. The trim therefore integrates measured-versus-specification, $10\log_{10}(z/S)$ — the quantity the test is judged on and the one Rattlesnake's own Response Error panel displays. It converges geometrically to $t = 1/b$, at which point the measurement sits on specification.

The solver-agreement gate, which is what stops the wind-up. Integrating measured-versus-specification on its own would wind up at every bin the plant cannot reach. This frame is rank 5 with six drives; at a structural null the law lands short of specification however much is asked of it, the error never clears, and a plain integrator would climb to its clamp demanding drive that buys nothing. So the trim is applied only where the solver reached what it aimed at — $|10\log_{10}(p/\text{target})| \leq \text{error_threshold_db}$, both sides cached from the cycle that issued the command, after the startup cap. A shortfall the solver itself predicted is an achievability limit and is none of this loop's business. Only the part the solver believed it had and did not get is model error.

How fast it acts. The per-channel part of the correction rides the ordinary scheduler: a changed target shows up in `_err_db` against the current H and step 3 picks those bins up worst-first, at the refinement budget's pace, which across 901 in-band bins is slow. The *common* part of each bin's change is therefore applied straight to the stored drive CPSD instead. Scaling X by a positive scalar scales $\text{diag}(H X H^H)$ by exactly that scalar, so the overall level correction — the dominant term, and the one an operator notices — is exact and immediate, and only the channel-to-channel shape waits for a re-solve. A real positive scalar leaves drive coherence and rank untouched, so this cannot walk around `max_drive_coherence` or `drive_rcond`.

Guards. A per-cycle clamp (`response_trim_step_db`, 0.5 dB) and a total authority clamp (`response_trim_limit_db`, ± 3 dB) bound it; a deadband (`response_trim_deadband_db`, 0.5 dB) keeps it off the frame-to-frame scatter of a short-average CPSD; scoring is in-band only, since out of band there is no drive energy and the measured diagonal is noise. Because `components/data_collector.py` divides every acquired frame by the *current*

test level, everything the law sees is referred to full level and the ratio is level-consistent in steady state — but not across a level change, where a frame spanning the ramp is normalised by the wrong number. A `set_test_level_db` hook, which Rattlesnake already calls on every change, stands the trim down for three cycles and discards the stale pairing. The accumulated trim is kept.

Measured offline (`verify_response_trim.py`, four tests). Against a plant 2.0 dB hotter than the model it was handed: on the channel-bins the solver reached, rms error 2.13 dB $\rightarrow$ 0.25 dB, with the trim settling at -1.76 dB and stopped by the deadband, in twelve updates. At provably unreachable channel-bins — two response channels given the same row of H and targets 10 dB apart, so the solver's own shortfall is a median 5.00 dB — the trim held at exactly 0.0000 dB. **Not yet run on the rig.**

WHAT THIS FIXES.
Everything above this loop
is
driven by H. Error the FRF
does not explain — H1
bias,
extraneous input, drive
clipping, a nonlinearity that
never appears in a linear
FRF — was invisible,
because
the law never looked at the
measured response. There
was no integral action, so
the offset simply stood.

control() is called
Rattlesnake passes
last_response_cpsd —
the response to the
command issued last cycle

response_trim_gain > 0 ?
default 0.0

yes
frozen after a
test-level change?

yes
decrement, drop the
cached pairing, return
a frame spanning the ramp
is divided by the
wrong test level and is not
comparable

no
z =
diag(last_response_cpsd)
p = prediction cached for
the command
actually issued, AFTER the
startup cap
t_c = the target that
command was solved
against

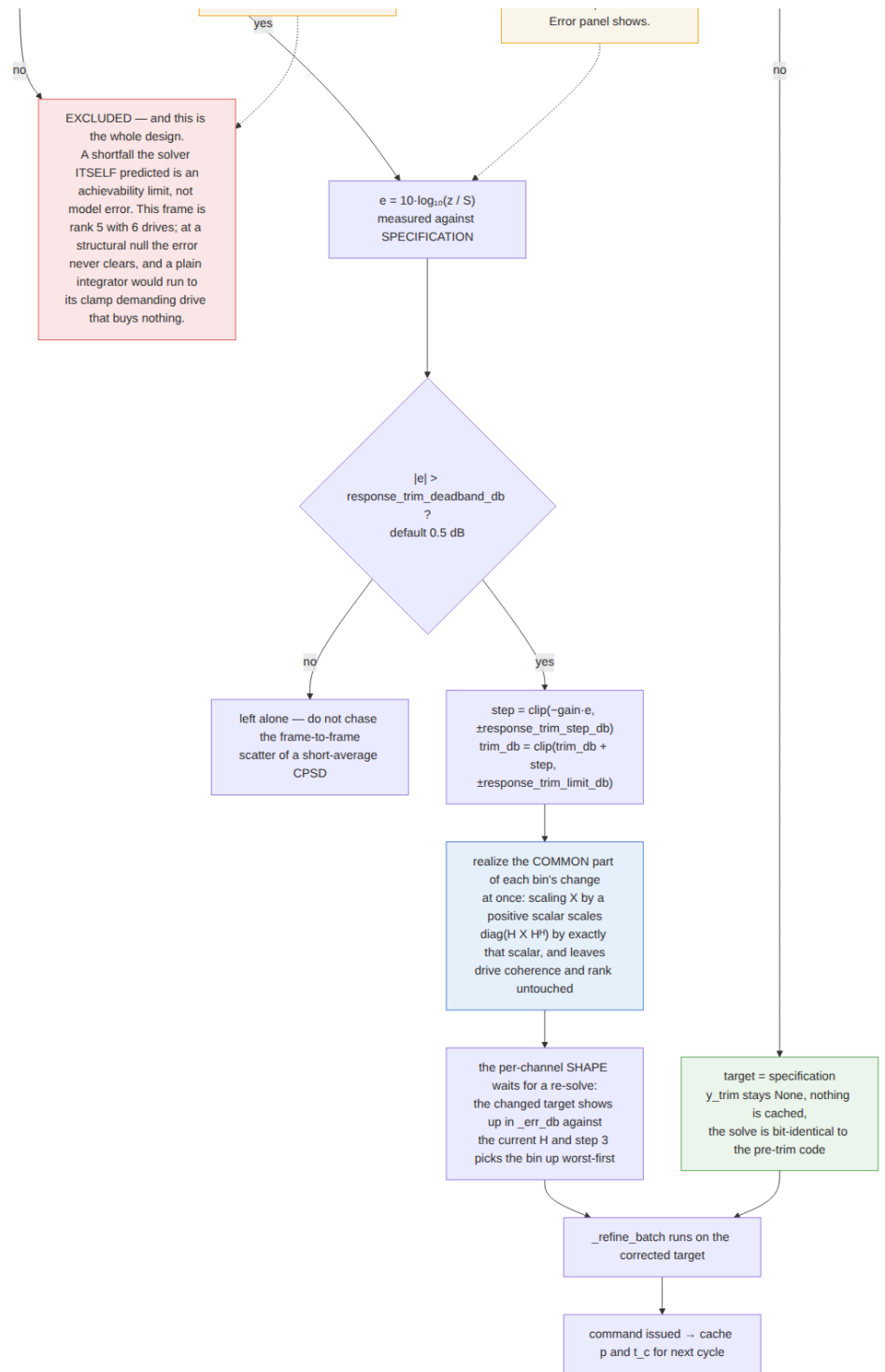
in band, and z, p, t_c, S all
> 0 ?

no
not scored
out of band there is no
drive energy, so the
measured diagonal is noise
and the ratio is unbounded

yes
SOLVER-AGREEMENT
GATE
 $[10 \cdot \log_{10}(p / t_c)] \leq$
error_threshold_db ?
did the solver get what it
asked for?

MEASURED OFFLINE.
Against a plant 2.0 dB
hotter
than the model it was
handed: on the channel-
bins the
solver reached, 2.13 dB
rms error → 0.25 dB, trim
settling at -1.76 dB and
stopped by the deadband,
in 12
updates. At provably
unreachable channel-bins
(two
response channels given
the same row of H and
targets
10 dB apart) the trim held
at exactly 0.0000 dB.

**WHY NOT measured-vs-
PREDICTED.** For a
constant
multiplicative plant bias b,
 $10 \cdot \log_{10}(z/p)$ reads
 $10 \cdot \log_{10}(b)$ for ever no
matter what the trim does
—
shrinking the target shrinks
prediction and
measurement
together. It is a model-
mismatch READOUT, not
an error:
an integrator driven by it
runs to its clamp and stops.
 $10 \cdot \log_{10}(z/S)$ drives to zero,
and is what the test is
actually judged on and
what Rattlesnake's own
Response



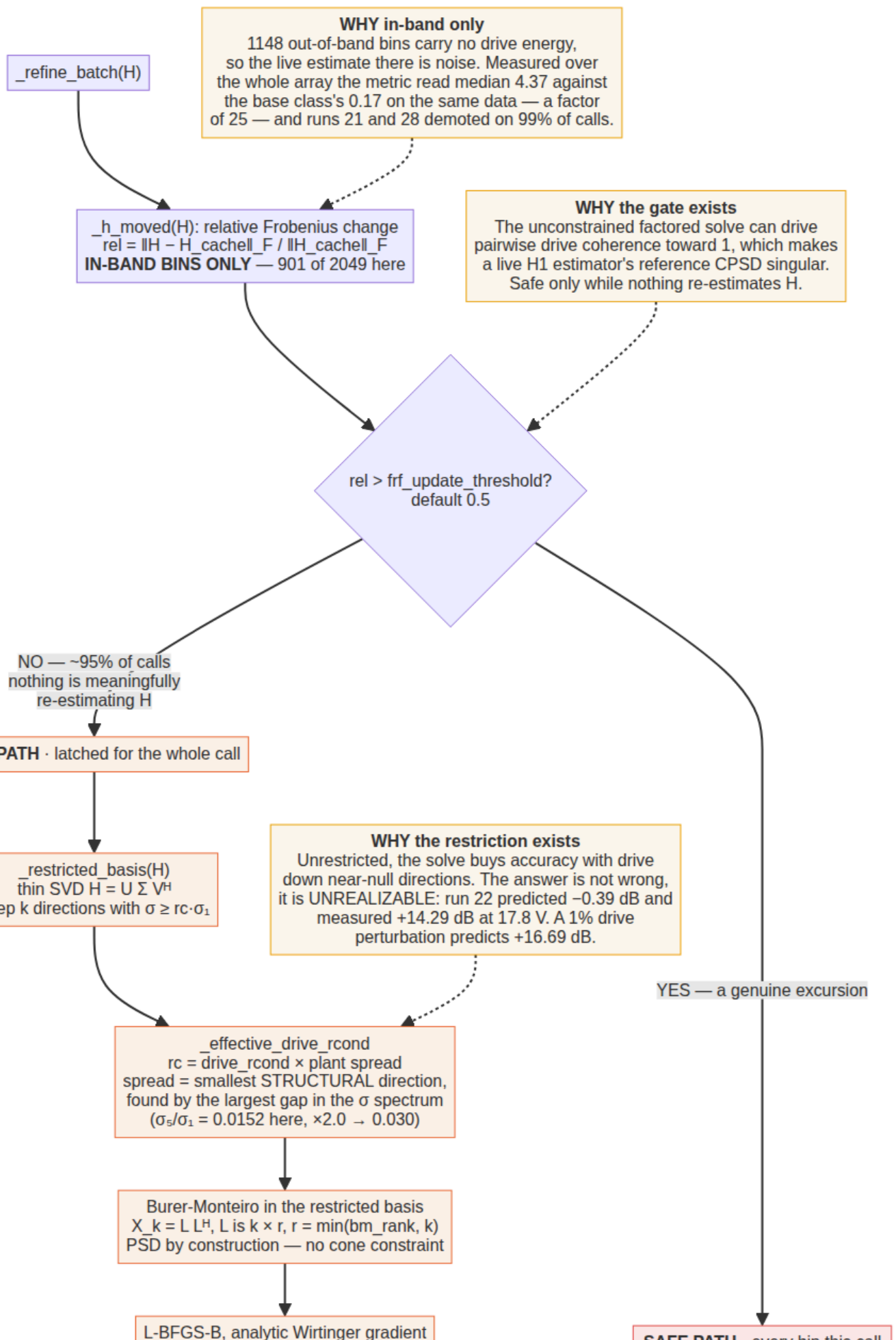
The response-error trim: which error signal, and the gate that stops it winding up at bins the plant cannot reach.

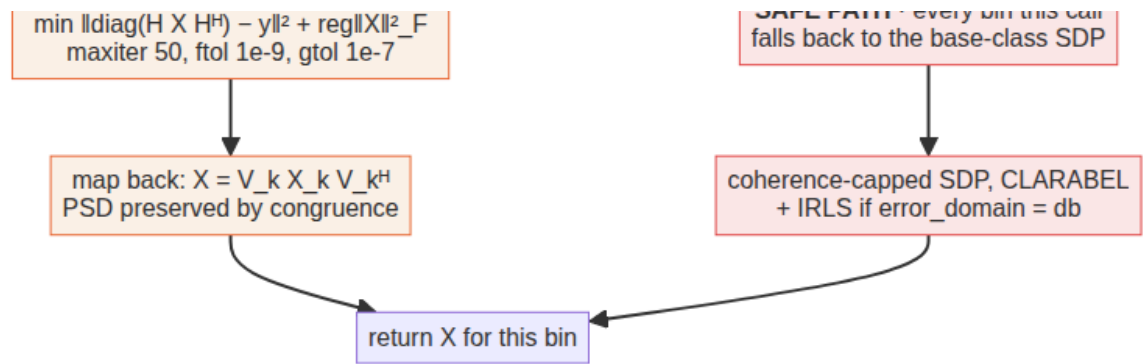
Solver failure

If the first SDP pass returns no solution, the bin falls back to a *diagonal-only* pseudoinverse synthesis, which silently abandons both the survey cross terms and the coherence cap, and the bin is still marked refined. A failure on a later IRLS pass keeps the last good solution and is not counted. `cum_solver_failures` in the log counts only the first kind; it read 0 on every healthy run in this campaign and 176 on run 30.

3. optimal_diagonal_control_fast — subclass, optimal_diagonal_control_fast.py:138

The same objective, solved by factorisation instead of by cone programming, when it is safe to do so.





optimal_diagonal_control_fast. The gate decides once per call; the restriction is what makes the factored answer realizable.

The factored solve

Write $X = L L^H$ with $L \in \mathbb{C}^{N \times r}$. Two things follow: X is PSD and $\text{rank}(X) \leq r$ by construction, so the PSD cone constraint disappears entirely rather than being enforced by an interior-point method; and the problem becomes smooth and unconstrained, which is what makes it fast. The objective is the base class's minus the coherence cap, which the factorisation cannot express:

$$\min_L \quad ||\text{diag}(H L L^H H^H) - y||^2 + \text{reg} * ||L L^H||_F^2$$

Optimised by L-BFGS-B with an analytic Wirtinger gradient $\nabla_L = 2(\partial f / \partial X)L$, verified against finite differences. The predicted diagonal is formed as $\sum_j |(HL)_{mj}|^2$, never by materialising $H X H^H$ — an $8 \cdot 6 \cdot r$ product instead of an $8 \cdot 6 \cdot 6$ chain, which is what makes each function evaluation cheap. Warm start is the top- r eigenpairs of the buzz solution for that bin.

Tolerances are deliberately loose: maxiter 50, ftol 1e-9, gtol 1e-7. At tight tolerances about 70 % of real bins hit a 200-iteration cap without visibly improving dB accuracy. Loosening cut time $5.7 \times$ — 18.7 to 3.3 ms per bin — for +0.4 dB mean and +2.7 dB worst-case against the SDP's own accuracy. **Net speedup on real data is about 3.6×**, much more modest than the 18–20× that synthetic-random-FRF benchmarks suggested, but honest.

The demotion gate

The unconstrained factored solve can drive pairwise drive coherence toward 1, which would make a live H1 estimator's reference CPSD matrix singular. That is fine if nothing is re-estimating H , and unsafe the moment something is. Rather than trying to detect the GUI checkbox — control()'s signature does not expose it — the law watches the safety-relevant property directly: **has H moved since the last call, by more than frf_update_threshold in relative Frobenius norm?** If it has, every bin for the rest of that call uses the base class's coherence-capped SDP. The decision is latched once per call.

This gate has been wrong twice, in two different ways First, it was bitwise equality, and the docstring claimed that with the FRF update off H is “set once and never re-passed”. That is not what Rattlesnake does: it passes a transfer function to control() every cycle in both regimes, and system_id_update() is never called during control at all. With the update off the array happened to be the same frozen array each time, so the fast path ran — the right outcome by the wrong mechanism. With the update on, every bin fell through to the SDP and the law was byte-for-byte the base class. **Second**, once it became a relative-change test, the norm was taken over the whole 2049-bin array. 1148 of those bins are out of band, where there is no drive energy, the live estimate is noise, and the relative change per cycle is unbounded. Measured over the whole array the metric read a median of 4.37 — a 437 % change every cycle — against the base class's in-band-only drift metric reading 0.17 on the same data. A factor of 25 between two measurements of the same quantity, and the reason runs 21 and 28 demoted on 99 % of cycles.

Masked to in-band bins, the measured per-cycle movement on this rig is median 0.196, 90th percentile 0.232, with a thin tail to 3.5. The default threshold was raised from 0.05 to **0.5** on 2026-09-19, which sits at about the 95th percentile: roughly 5 % demotion. At 0.05 the gate was below essentially the entire distribution, so it was always open — and because the same parameter drives the base class's drift step, every previously refined bin also counted as drifted, so half the refinement budget went to re-solving old bins forever. Changing the default altered no archived run: every profile spreadsheet sets the field explicitly.

drive_rcond — the realizability constraint

The factored path has no coherence cap and, before 2026-09-18, nothing else constraining it either. Left unconstrained it buys accuracy with drive pushed down near-null directions of H . The resulting solution is **not wrong — it is unrealizable**: its accuracy lives in cancellation between drives that survives only if the drive is reproduced to a precision no rig delivers.

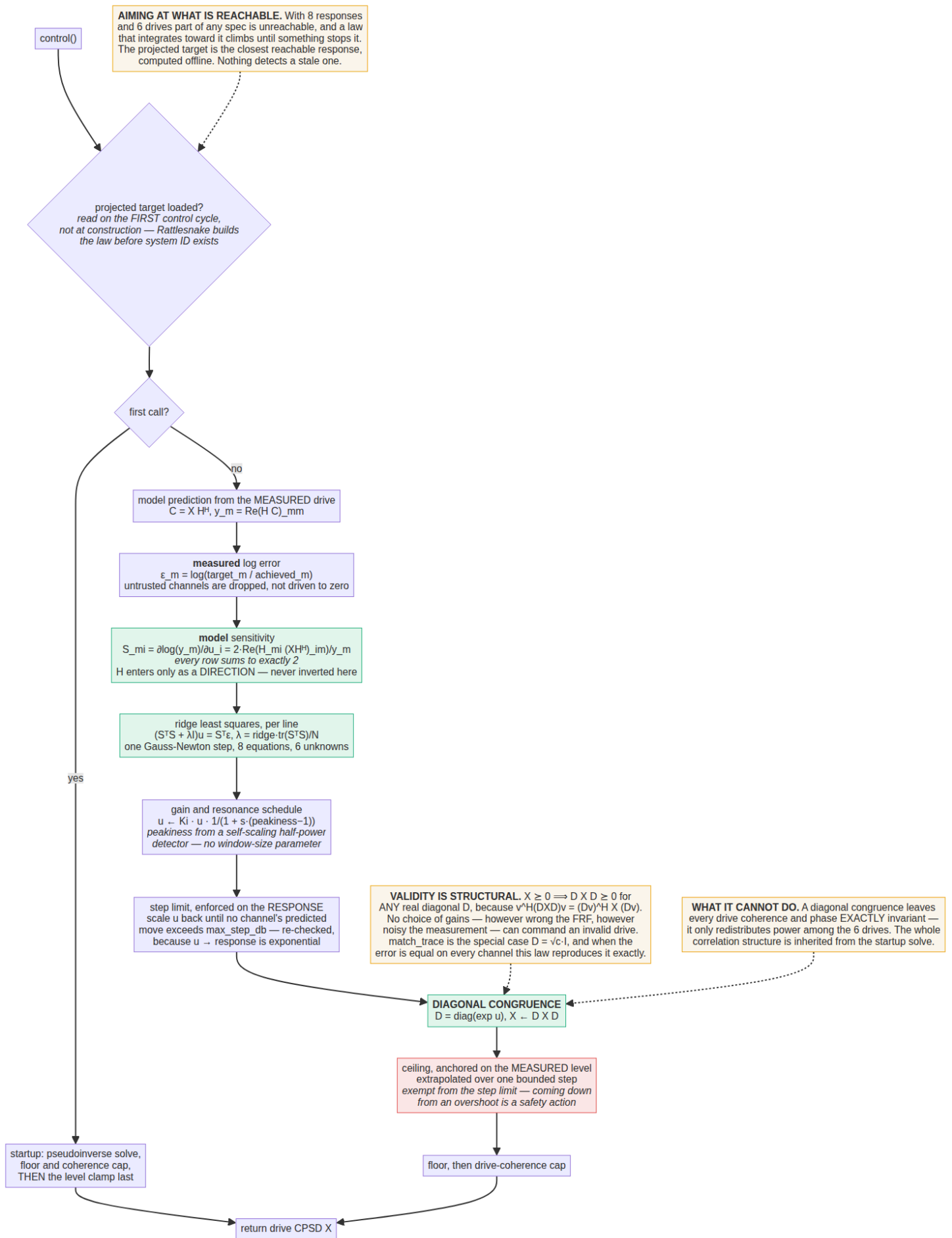
_restricted_basis takes the thin SVD of the bin's H , keeps the k right-singular directions with $\sigma \geq rc \cdot \sigma_1$, solves in that basis and maps back by congruence, so the drive is forbidden from having any component along the plant's weakest directions.

`_effective_drive_rcond` derives `rc` from the plant itself. A positive `drive_rcond` is read as a *multiple* of the smallest **structural** singular direction; a negative value is an absolute ratio. Finding “structural” is the interesting part: this frame is rank 5, and its measured in-band median ratios run 1.000, 0.319, 0.084, 0.032, 0.015, 0.0017. That last value is the identification noise floor, not a direction the plant has, so scaling off $\sigma_{\min}$ would track how good the system ID was rather than how the structure is conditioned. The reference is instead found by **the largest gap in the spectrum** — $8.9\times$ between σ_5 and σ_6 here, against $2.1\text{--}3.8\times$ everywhere else — and taking the smallest direction above it. Default multiple 2.0 gives 0.030 on this frame, which is the value the rig sweep found independently.

The same out-of-band mistake, a third time The singular-value spread was also first measured over all lines, which moved the reference from 0.0152 to 0.1435 — a factor of 9, the difference between the tuned $3.0e-2$ and a useless 0.29. Out-of-band lines are not controlled and are conditioned quite differently. The rule now stated in the code is: *any statistic taken over the FRF array on this rig must be masked to in-band bins*. It has been got wrong three times — the singular-value spread, the gate metric, and the divergence guard — each written after diagnosing the previous one.

4. `match_diagonal_congruence` — class, `diagonal_congruence_control.py`:233

`match_trace`'s one scalar, generalised to N independent knobs, with the validity guarantee kept. The only genuine feedback law in this document besides `match_trace`.



match_diagonal_congruence. Measured error, modelled sensitivity, and a correction that cannot produce an invalid drive.

The idea

match_trace multiplies the whole drive CPSD by one positive scalar. That is why it is robust — a uniform positive scalar can never turn a valid drive CPSD into an invalid one — and why it is limited: if channel 3 is 5 dB high and channel 5 is 5 dB low, the trace is correct and the law does nothing at all. Replace the scalar with a **diagonal congruence**:

```
X <- D X D,    D = diag(exp(u_1) ... exp(u_N)),    u real
```

Congruence preserves positive semidefiniteness structurally: for any vector v , $v^H(DXD)v = (Dv)^H X (Dv) \geq 0$ whenever $X \geq 0$, because D is real diagonal. There is no condition on D at all — the gains can be arbitrarily wrong, the FRF can be badly estimated, and the commanded CPSD is still a physically realizable drive. What is gained over match_trace is N independent knobs instead of one, which is what per-channel correction requires. **match_trace is the special case $D = \sqrt{c} \cdot I$** , and the degeneracy is exact: every row of the sensitivity matrix sums to 2, so when the error is the same on every channel the solution is $u_i = \epsilon/2$ for all i — the law reproduces match_trace's update exactly and degrades gracefully to the law it generalises.

The per-line step

Everything is a batched solve over the frequency axis — 2049 independent 6×6 solves per cycle, no Python loop over lines.

1. **Measured log error** $\epsilon_m = \log(\text{target}_m / \text{achieved}_m)$, natural log, from the diagonal of the measured response CPSD. A channel measuring non-finite or non-positive this cycle is dropped from the solve rather than driven to zero.
2. **Model sensitivity** $S_{mi} = \partial \log(y_m) / \partial u_i = 2 \cdot \text{Re}(H_{mi}(XH^H)_{im}) / y_m$, an 8×6 real matrix per line, where y is the *model-predicted* response. Note the hybrid: the error is measured, the sensitivity is modelled.
3. **Ridge least squares** $(S^T S + \lambda I)u = S^T \epsilon$ with $\lambda = \text{ridge} \cdot \text{tr}(S^T S) / N$ — scale-relative, so a line with tiny FRF gain gets the same *relative* regularisation as a strong one. Eight equations, six unknowns; the solution is the best 6-DOF fit to the 8-channel error.
4. **Gain and resonance schedule.** $u \leftarrow K_i u$ times a per-line factor $1 / (1 + s(\text{peakiess} - 1))$, because a resonance carries dynamical phase lag and has a much lower safe gain than a flat line.
5. **Step limit, enforced on the response.** Clamping u itself does *not* bound the response move: each row of S sums to 2, but individual entries can be large and of opposite sign, so a bounded gain vector can still swing one channel a long way. The limit is therefore applied to the exact predicted per-channel log response change, with u scaled back and re-checked up to six times because the map from u to response is exponential, not linear.
6. **Command, ceiling, floor, coherence cap.**

H enters only as a direction

In the feedback path H is never inverted — it appears only in forming the sensitivity, so a mediocre FRF still yields a usable direction and the feedback cleans up the rest. That is precisely what a pinv-based law cannot do, and it is where the earlier drive-imbalance events came from. The only pseudoinverse in this law is in the startup solve.

The ceiling is the anti-windup, and it is deliberately asymmetric

Two estimates of the response level are formed. The **measured-anchored** one is $\text{achieved} \times \exp(\text{move})$: the absolute level comes from the measurement and is therefore model-free, with only the small, already-bounded step extrapolated through H , so even a badly wrong FRF can distort it by at most max_step_db. The **model-predicted** one is used only where this cycle's measurement cannot be trusted, and its net is set 20 dB looser.

Why the measurement replaces rather than combines Where the measurement is trustworthy it *replaces* the model estimate. Taking the stricter of the two looks conservative but is not: a model that *overestimates* the response then holds the drive down forever and the test never reaches specification — a functional failure dressed up as caution. The 20 dB model margin exists because “the FRF is wrong by 30 dB” and “a control accelerometer has failed low” produce identical evidence, and no amount of cleverness distinguishes them from those two signals alone. A choice is forced, and this is it.

The ceiling is exempt from the step limit: coming down from an overshoot is a safety action and happens in one cycle, not over the many cycles a step limit would impose.

Aiming at what is reachable

The law optionally accepts a **projected target** — the closest reachable response, computed offline by achievable_response.achievable_diagonal — and drives to that instead of the raw specification, because integrating toward an unreachable target is how the earlier resolve laws reached their anti-windup ceilings. The file is read on the *first control cycle*, not at construction, because Rattlesnake builds the law before system identification runs so the projection cannot exist yet. Every failure mode — no path, missing key, unreadable, wrong shape — falls back to the raw specification with a message and never raises. **A stale projection is not detected.** It is tied to the FRF it was computed from and must be regenerated after a new system ID; nothing checks this.

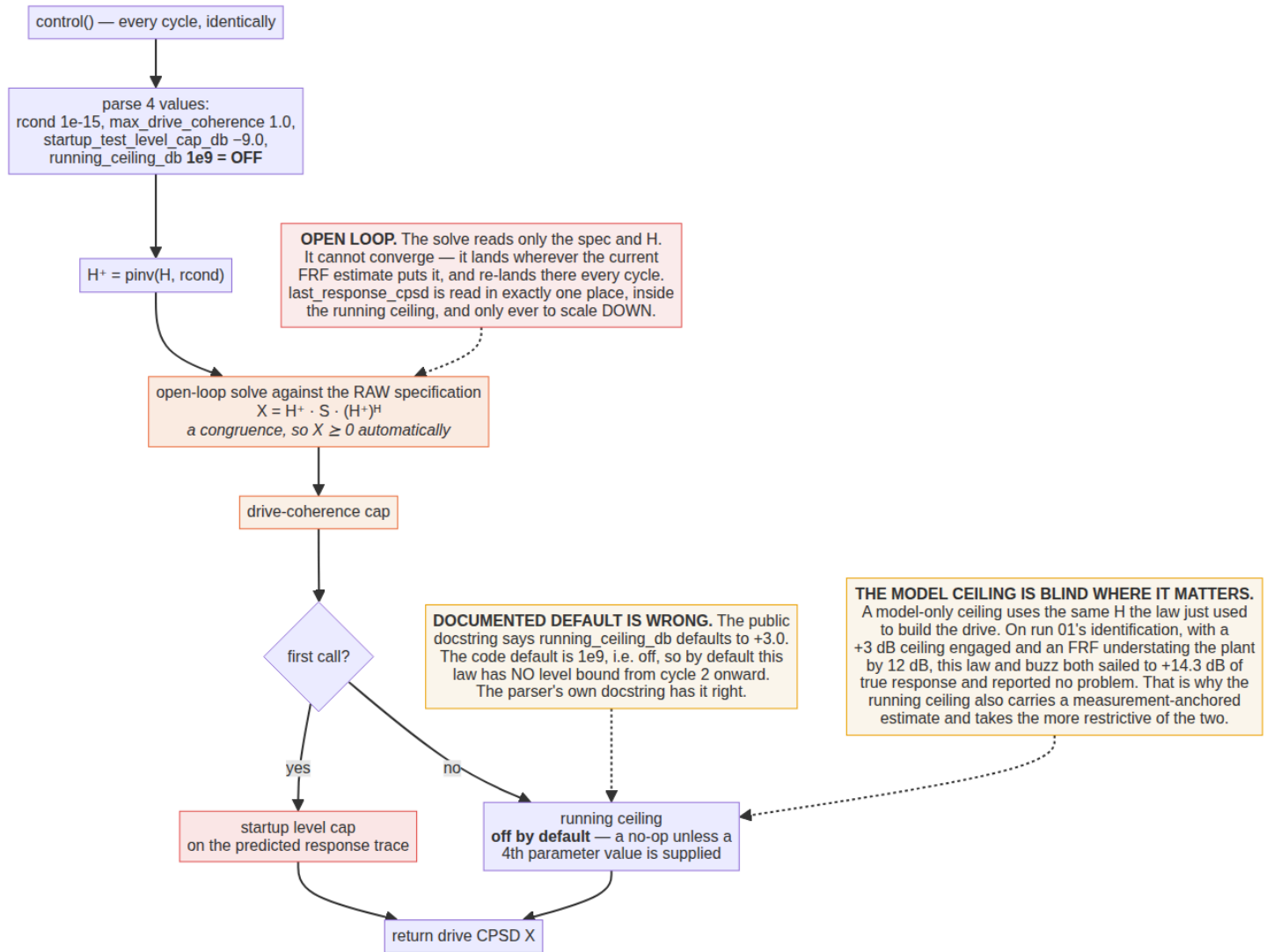
What it cannot control

A diagonal congruence leaves drive coherence and drive phase *exactly* invariant: $|D_i X_{ij} D_j| / \sqrt{(D_i^2 X_{ii} \cdot D_j^2 X_{jj})}$ reduces to the original coherence. The law can only redistribute power among the six drives; the entire correlation structure is inherited from the startup solve. It also does not touch response cross-spectra — the error is built only from the diagonal — and it reads no coherence or frame-count information at all.

An inconsistency worth knowing about The startup path's own comment states the principle that the safety ceiling must go last, because the floor and the coherence cap can both raise level back through it. In the feedback path the order is the opposite: ceiling, then floor, then coherence cap. No comment explains the asymmetry. It may be benign — in the feedback path the incoming drive is already capped, so the cap is close to a no-op there — but it is not documented either way.

5. pseudoinverse_control — function, control_laws.py:248

The bare least-squares solve, recomputed identically every cycle.



`pseudoinverse_control`.

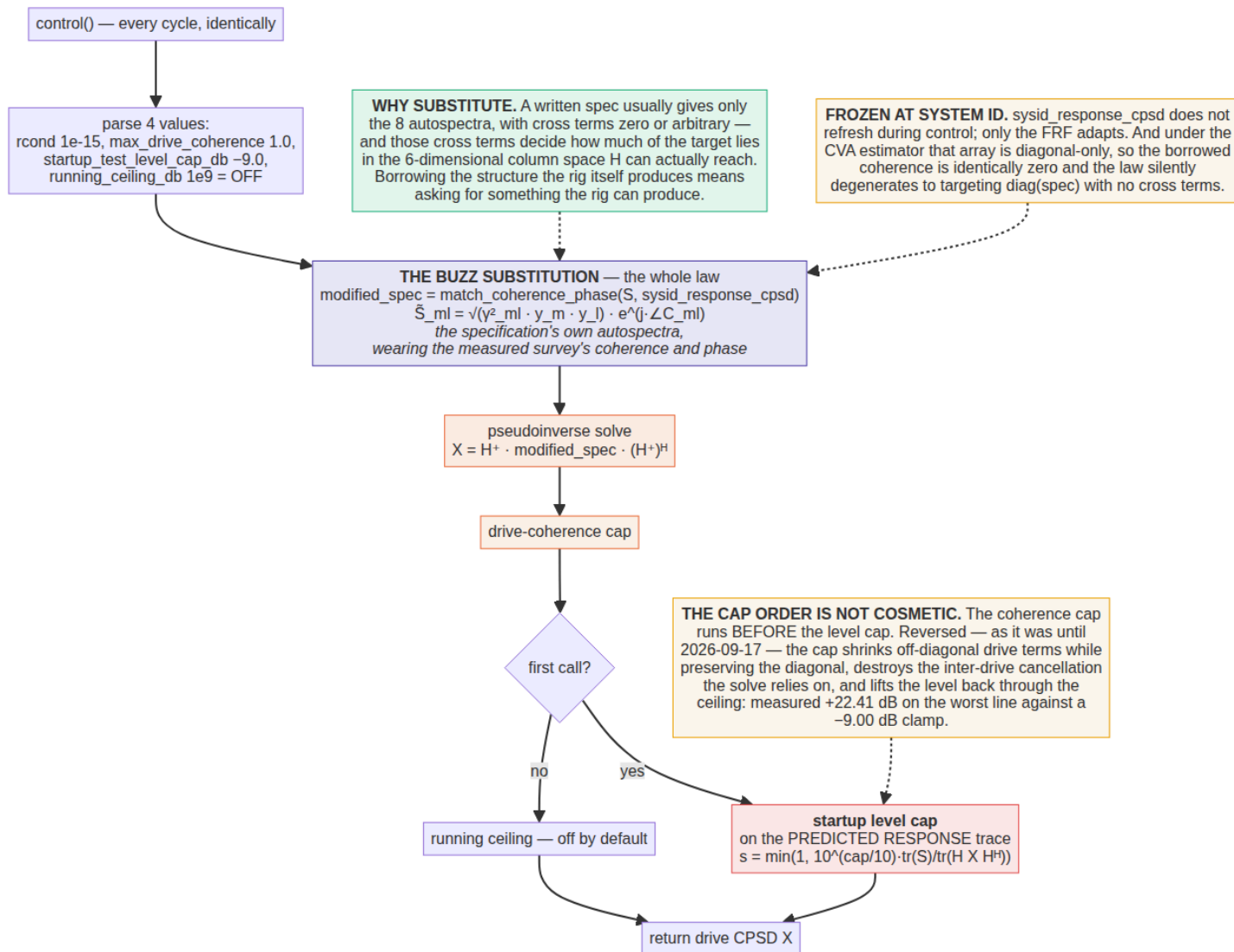
$X = H^+ G_{xx} (H^+)^H$, a congruence so X is PSD automatically, with the predicted response being the specification projected onto the column space of H . The solve reads only the specification and the FRF. It never reads the measured response, has no state, and **cannot converge** — it lands wherever the current FRF estimate puts it and re-lands there every cycle. With the FRF update on it tracks the plant *model*, never its own error.

Two documentation defects in this law, both live The docstring says `running_ceiling_db` defaults to +3.0. **The code default is 1e9 — off.** So by default this law has no level bound at all from cycle 2 onward. The parser's own docstring has it right and explains the reason: a default-on ceiling would have clamped runs 09 and 11 and made the twelve-run comparison set unreproducible. Separately, the Notes claim `last_response_cpsd` is “accepted and never referenced”. Since the running ceiling was added it *is* referenced — but only to scale down, so the substance of the claim (not closed loop) survives while the literal wording does not.

The running ceiling, when switched on, carries the same measurement-anchored estimate the congruence law uses, for the same reason: a model-only ceiling uses the same H the law just used to build the drive, so it is blind in exactly the direction that hurts. On run 01's identification, with a +3 dB ceiling engaged and an FRF understating the plant by 12 dB, this law and buzz both sailed to +14.3 dB of true response and reported no problem.

6. buzz_control — function, control_laws.py:1183

pseudoinverse_control with the target's cross terms replaced by the ones the rig actually produces. One line of difference, and it is worth 1.1 dB and a third of the drive.



`buzz_control`. One substituted line is the whole law.

Per line, build a new target whose *diagonal* is the specification's and whose off-diagonal structure is copied wholesale from the measured system-identification response:

```
S~_ml = exp(i*phi^sysid_ml) * sqrt( coh^sysid_ml * S^spec_mm * S^spec_ll )
```

then solve $X = H^+ \tilde{S} (H^+)^H$ exactly as `pseudoinverse_control` does. A written specification usually gives only the eight autospectra, with cross terms zero or arbitrary — and those cross terms decide how much of the target lies in the six-dimensional space H can reach. Borrowing the structure the plant naturally generates means asking for something the plant can generate.

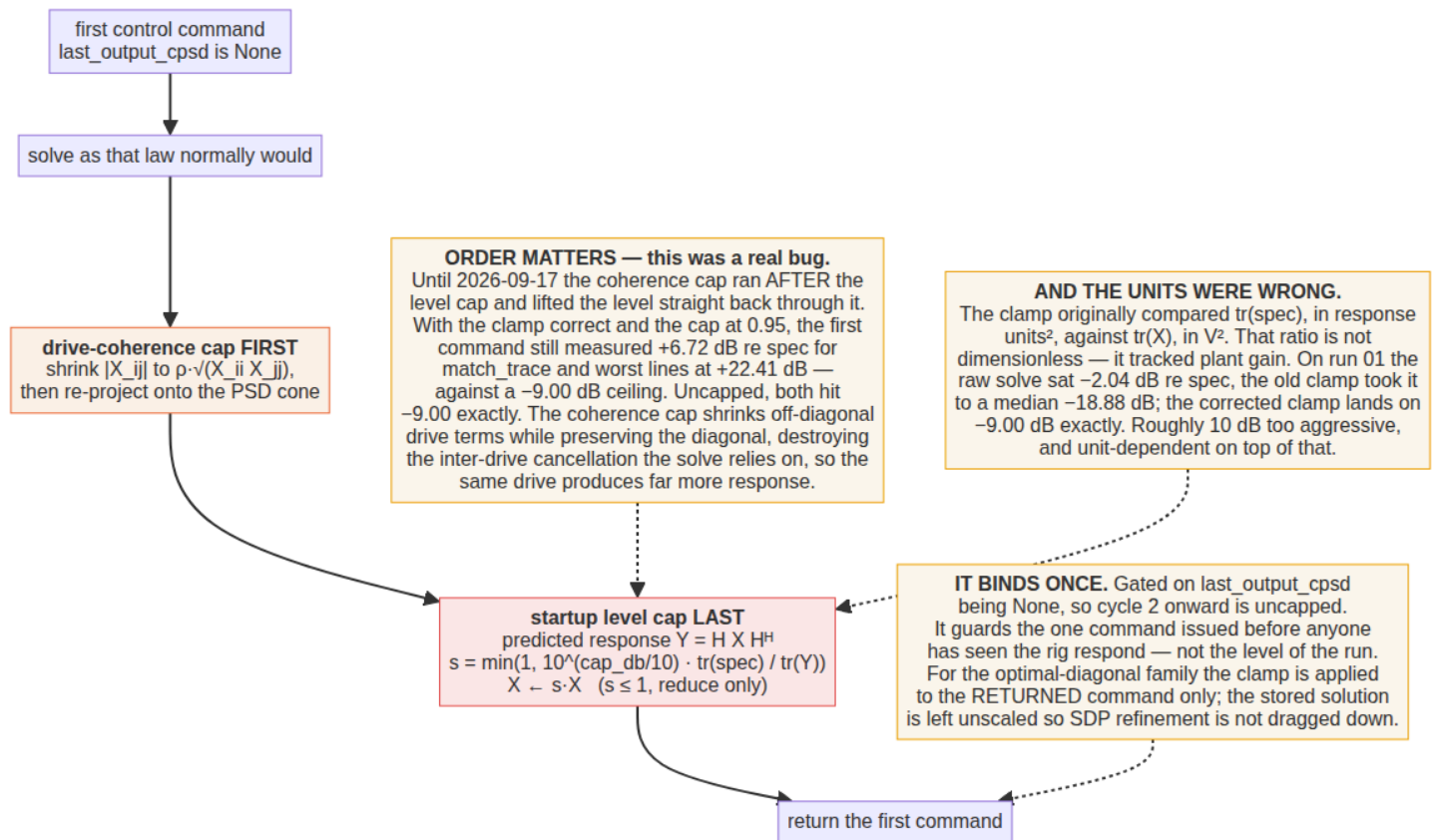
Two failure modes to know about The survey is frozen at system identification and does not refresh during control; only the FRF adapts. And under the CVA estimator, `sysid_response_cpsd` is diagonal-only with zero off-diagonal terms — so the borrowed coherence is identically zero and **buzz silently degenerates to targeting the bare diagonal with no cross terms**, i.e. to something close to plain pseudoinverse. This is documented in the spectral-processing code, which names `match_coherence_phase` explicitly.

Also worth knowing: only the *function* form carries the safety machinery. The generator and class variants of `buzz` ignore `extra_parameters` entirely — no `rcond`, no coherence cap, no startup cap, no running ceiling. If a run is configured with `control_python_function_type` 1 or 2, none

of the guards in §7 exist.

7. The startup level cap and the drive-coherence cap

On the first control cycle no measured response exists, so a law falls back to a raw solve on whatever FRF the spectral processing has produced — which on the first frame is a single noisy unaveraged estimate under both exponential and linear averaging, because there is no minimum-frames gate before an FRF is published and handed to the control law. On real hardware this was observed to overshoot by several decades.



The startup cap and the ordering it depends on.

What the cap does

Per frequency line, with the predicted response $Y = H X H^H$:

$$s = \min(1, 10^{(\text{cap_db}/10)} \cdot \text{tr}(\text{spec}) / \text{tr}(Y)), \quad X \leftarrow s \cdot X$$

Both sides are response-domain powers, so the ratio is dimensionless. The $\min(1, \cdot)$ makes it strictly a reduction — a positive cap value does not raise a command that is already low. Default -9.0 dB; 0 dB means full spec-match. It is disabled by a large value, and the implementation deliberately uses a numpy float rather than a Python one so that an absurd value like $1e6$ gives $+\text{inf}$ rather than raising `OverflowError`.

It binds on the first command only, in every law that has one, gated on `last_output_cpsd` is `None`. For the optimal-diagonal family the clamp is applied to the *returned* command while the stored solution is left unscaled, so the SDP refinement is not dragged down with it.

The two bugs, both silent

Units. The clamp originally compared the specification trace, in response units squared, against the *drive* trace, in volts squared. That ratio is not dimensionless; its value tracked plant gain and response-channel units. On run 01's identification the raw solve sat at -2.04 dB re specification, the clamp as written took it to a median -18.88 dB, and the corrected clamp lands on -9.00 dB exactly — roughly 10 dB too aggressive, and unit-dependent on top of that.

Order. Both callers ran the coherence cap *after* the level clamp, which lifted the level straight back through it. Measured with the clamp correct and the cap at 0.95 , the first command came in at $+6.72$ dB re specification for `match_trace` and -0.12 dB for buzz, with worst single lines at $+22.41$ and $+16.53$ dB, against a -9.00 dB ceiling. Uncapped, both hit -9.00 exactly. The mechanism: the coherence cap shrinks off-diagonal drive terms while preserving the diagonal, which destroys the inter-drive cancellation the pseudoinverse solve relies on, so the same drive produces far more response.

The drive-coherence cap itself

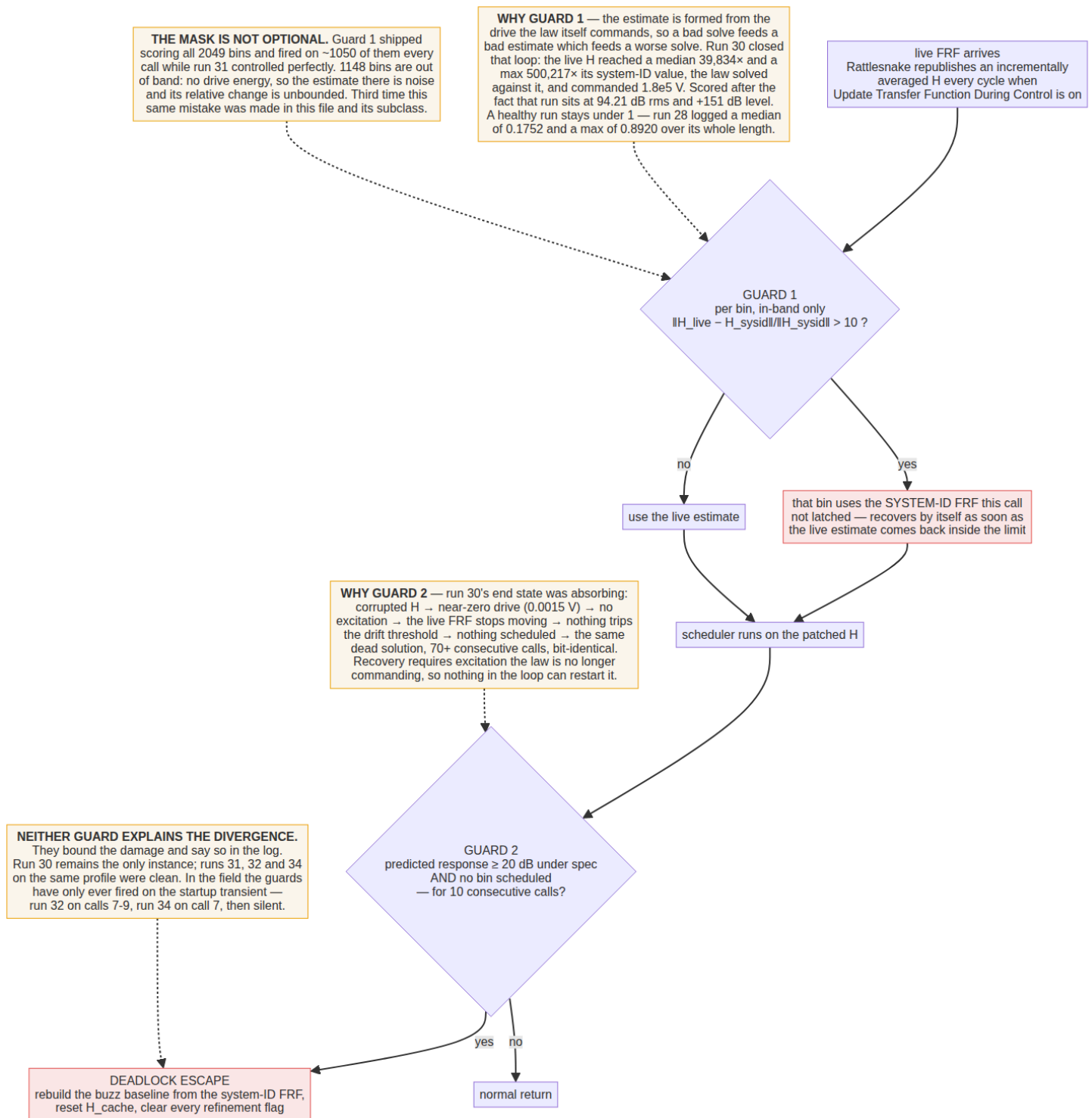
For each line and each of the 15 drive pairs, if $|X_{ab}|/\sqrt{(X_{aa}X_{bb})}$ exceeds the cap, the cross term's magnitude is shrunk to the cap while its phase is preserved exactly and both diagonals are left untouched; if anything changed, the matrix is re-projected onto the PSD cone by clipping negative eigenvalues. Note the comparison is against the *magnitude* coherence, not the squared one: 0.95 means $\gamma = 0.95$, i.e. $\gamma^2 = 0.9025$. A value of 1.0 or more is an explicit no-op that returns the input unchanged.

The re-projection is why the cap is **not idempotent** and why it can move the level — the fact that drives the ordering rules above.

The cap means two different things The optimal-diagonal family solves *subject to* the cap, as a constraint inside the SDP, so the solution is optimal given the constraint. Every other law post-processes it onto a finished solve, which is a projection, not an optimisation — it takes a solution that depended on particular cross terms and edits those cross terms. That distinction, not the loop type, is what predicts whether the cap costs anything.

8. The two live-FRF guards

Added 2026-09-19 to `optimal_diagonal_control` and therefore inherited by `optimal_diagonal_control_fast` and `octave_band_switching_control`. **Not** inherited by `match_diagonal_congruence` (a standalone class) or `match_trace_pseudoinverse` (a plain function).



The two live-FRF guards.

Guard 1 — refuse an FRF that is no longer a plant model

When the FRF update is on, Rattlesnake republishes an incrementally averaged estimate every cycle — and that estimate is formed from the drive the control law itself commands. A bad solve therefore feeds a bad estimate which feeds a worse solve. Run 30 closed that loop: the live FRF reached a median 39,834× and a maximum 500,217× its system-identification value, the law solved against it, and commanded 1.8×10^5 V.

The guard tests, **per bin and in band only**, $\|H_{\text{live}} - H_{\text{sysid}}\| / \|H_{\text{sysid}}\| > 10$. A failing bin reverts to its system-ID value *for that call*. It is not latched and recovers by itself as soon as the live estimate comes back inside the limit. Per bin rather than globally, so one wild line cannot condemn the whole array and the law keeps controlling on the bins that are still sane.

The limit of 10 has an order of magnitude of headroom over anything measured on a working run — run 28 logged a median of 0.1752 and a maximum of 0.8920 over its whole length — and still catches run 30 at its 55th call.

Guard 2 — break out of the zero-drive absorbing state

Run 30's end state could not recover on its own: corrupted H produced a near-zero commanded drive (0.0015 V), which meant no excitation, so the live FRF stopped moving, so nothing tripped the drift threshold, so no bins were scheduled, so the law kept commanding the same dead solution — seventy consecutive calls with every scheduler field zero and a bit-identical self-prediction. Restarting requires excitation the law is no longer commanding.

Detection is deliberately narrow: the predicted response trace must be 20 dB below the specification trace *and* no bin may have been scheduled, for ten consecutive calls. A working run sits within a few dB of target, so this cannot fire on one. Recovery discards the corrupted state entirely and rebuilds the buzz baseline from the system identification — the one plant model known to be good.

What the guards are not Neither guard explains what made the estimate diverge; they stop the law amplifying it and say so in the log. The demotion gate, notably, *worked* throughout run 30 — demoting 69 % of calls — and did not help, because falling back to the safe solver cannot rescue a solve when the plant model is wrong: both solvers get the same bad H. The gate protects against solver fragility, not estimator divergence.

Field record to date. Verified offline that on ordinary 3 % averaging jitter neither guard fires and the control output is bit-identical to the pre-guard code. On the rig they have only fired on the startup transient: run 32 on calls 7, 8 and 9 covering 4, 20 and 3 in-band bins, and run 34 on call 7 covering 15. Two different laws, the same call numbers, the same handful of bins — that excursion is a property of the rig's startup, not of either law. Run 30 remains the only divergence and its trigger is unexplained.

9. achievable_response — a diagnostic, not a control law

Answers the question every other section depends on: *how much of the response error is the control law's fault, and how much is simply not reachable with this structure and these shakers?*

achievable_diagonal(H, y_target)
not a control law — no hardware

per frequency line, over $X \geq 0$

Burer-Monteiro factorisation
 $X = L L^H$, L is $N \times K$
PSD structural, parametrisation unconstrained

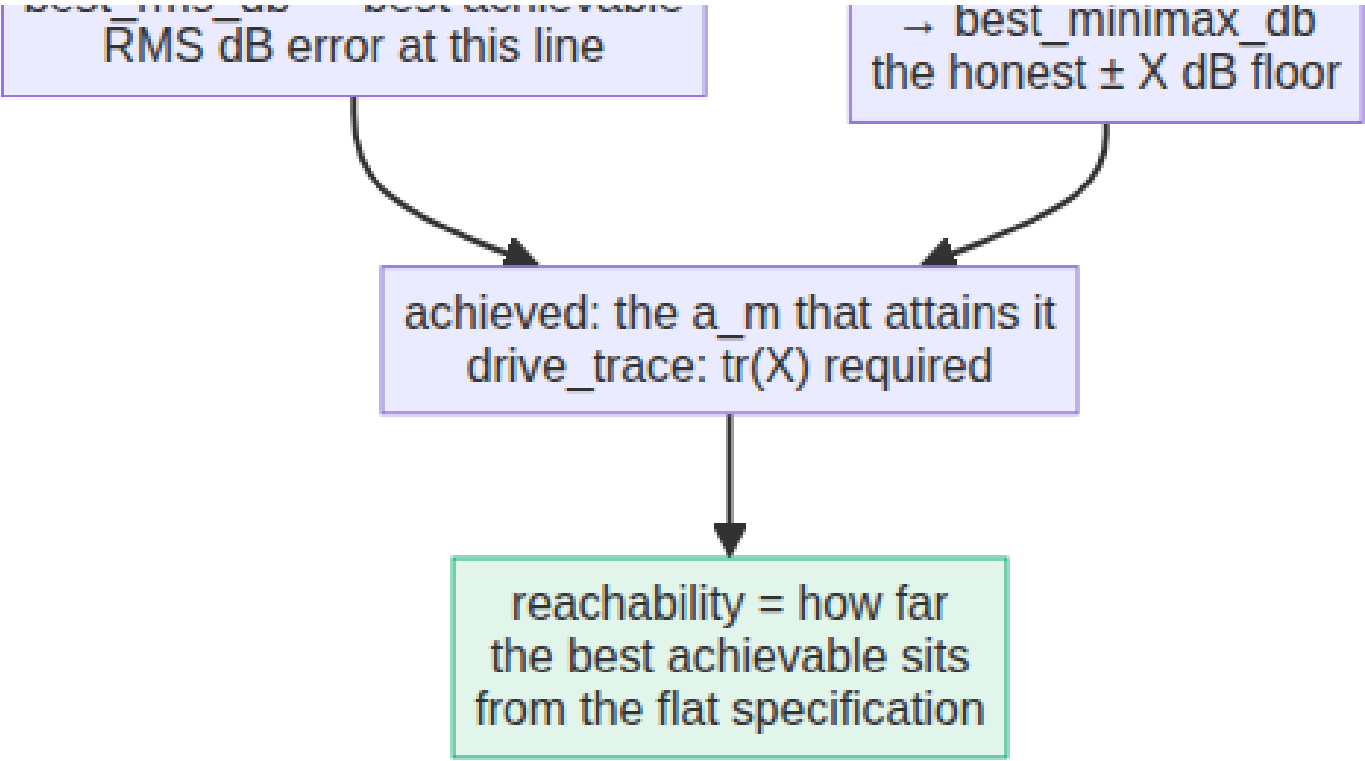
seed: pseudoinverse solve
plus n_{restarts} random starts

least_squares (TRF) on the dB residual
 $d_m = \log(a_m) - \log(y_m)$
 $a = \text{diag}(H X H^H)$

Wirtinger Jacobian
 $\partial a_m / \partial L = \text{conj}(h_m) \otimes r_m / a_m$
split into Re and Im blocks

best rms dh — best achievable

softmax sweep over β



achievable_response: the reachability floor.

Per line it minimises, over $X \geq 0$, the dB error between $\text{diag}(H X H^H)$ and the target, returning both a best RMS and a best minimax figure — the latter being the honest $\pm X$ dB floor. Errors are in dB rather than as a linear residual deliberately: a linear residual, which is what the SDP minimises, implicitly weights the loudest channels hardest, and on a spec with any dynamic range across channels that is not the quantity the test is judged by.

It uses the same Burer-Monteiro factorisation as the fast law, with the *pseudoinverse solution as its first starting point* — the solution the existing laws use — so the floor search starts from the incumbent answer and can only improve on it. It also returns drive_trace, which turns “mathematically reachable” into “reachable with these shakers”.

restrict_rcond is the parameter that makes the floor comparable to a real law: it confines the drive to the retained right-singular subspace, so the floor can be computed under the same truncation the law actually uses. This separates *how much accuracy is lost by discarding near-singular directions*, a deliberate safety choice, from *how much the law is simply leaving on the table*.

The floor is not a constant — measured 2026-09-20 Computed at restrict_rcond = 1e-3 over all 31 runs, the floor is not a property of the rig: it tracks the identification. On most runs the sixth singular value sits near 1e-4, well below the cutoff, and the floor lands between 3.13 and 3.92 dB. On runs 15, 23 and 26 the identification lifted σ_6/σ_1 above 1e-3, the solver got a sixth drive direction, and the floor collapsed to 1.83–2.14 dB. **“Error against the floor” is therefore not comparable across runs whose identifications differ in quality.** The sixth singular value is identification noise, not structure — which is exactly why drive_rcond locates the structural floor by gap detection instead.

One detail worth knowing before consuming the output: achieved, drive_trace and drive_cpds all come from the RMS-optimal point, while best_minimax_db comes from a different point whose drive is not returned. A consumer projecting a specification onto achieved — as the congruence law’s projected target does — is projecting onto the RMS optimum.

10. Parameter reference

Every law parses its parameters positionally from one comma-separated string, and every parser swallows a malformed field and falls back to the default **without warning**. A typo degrades silently. Note also that position 3 means different things to different law families, which is why match_trace deliberately skips it.

match_trace_pseudoinverse · control_laws.py:520

pos	name	default	meaning and extremes
0	rcond	1e-15	pinv truncation, relative to sigma_1. At the default nothing is truncated, so near-null directions are inverted with enormous gain.
1	max_drive_coherence	1.0	Pairwise drive coherence cap, magnitude not squared. 1.0 or more is an explicit no-op.

pos	name	default	meaning and extremes
2	startup_test_level_cap_db	-9.0	Power dB re the specification trace, first command only. Disable with a large value.
3	(skipped)	—	Deliberately unread, so that a string written for the optimal-diagonal laws cannot be copied here and silently mean something else.
4	refresh_shape	off	Re-solve the drive shape from the live FRF each cycle. Off by default, which is what makes the FRF update a no-op for this law.

optimal_diagonal_control and _fast · optimal_diagonal_control.py:234

pos	name	default	meaning and extremes
0	reg	1e-6	Tikhonov weight on $\ X\ _F^2$ in both the SDP and the factored objective. Frozen into the compiled problem at first build; changing it later is a silent no-op.
1	frf_update_threshold	0.5	Dual use: the base class's drift step, and the fast subclass's demotion gate. Raised from 0.05 on 2026-09-19. 0 restores any-change behaviour.
2	max_bins_per_update	20	SDP solves per call. The drift step is capped at half of it. 0 means no bin is ever refined and the law stays at the buzz baseline forever.
3	error_threshold_db	1.0	Two roles: the bar a new bin must exceed to be worth refining, and the degradation margin for the stale re-triage.
4	max_drive_coherence	0.95	SDP constraint, not a post-process. 1.0 disables the constraint entirely. Has no effect on the fast path, which is unconstrained by design. Frozen at first build.
5	bm_rank	4	Factorisation rank for the fast solve, effective value $\min(\text{bm_rank}, N)$. Read only by the subclass; the base class parses past it so one string serves both.
6	startup_test_level_cap_db	-9.0	As above. Applied to the returned command only; the stored solution stays unscaled.
7	error_domain	linear	0 is linear, anything else is dB. Linear reproduces runs 03-06 exactly. dB gives better pooled rms but spreads error across every channel instead of sacrificing the two the plant cannot reach.
8	n_irls_passes	2	Extra reweighted passes in dB mode. No effect on the fast path, which reaches the dB objective directly.
9	drive_rcond	2.0	A MULTIPLE of the plant's structural singular-value spread. Negative means an absolute ratio. 0 disables the restriction and recovers run-22 behaviour. Factored path only.
10	response_trim_gain	0.0	Fraction of the measured response error folded into the target each cycle. 0 is OFF and is the default: the trim array is never allocated and the command is bit-identical to the pre-trim code. 1.0 is deadbeat on the measurement and is not recommended against a short-average CPSD.
11	response_trim_limit_db	3.0	Total authority of the trim, plus or minus. The hard bound on how far the target can be moved away from the specification.
12	response_trim_step_db	0.5	Most the trim may move in one cycle. This, not the gain, is what bounds the response to a single bad frame.
13	response_trim_deadband_db	0.5	Measured errors smaller than this are left alone. 0 makes the trim chase CPSD scatter.

match_diagonal_congruence · diagonal_congruence_control.py:174

pos	name	default	meaning and extremes
0	rcond	1e-3	Startup pseudoinverse only. Not used in the feedback loop at all.
1	max_drive_coherence	0.95	Defaults to 0.95, not 1.0: uncapped drive coherence lets shakers fight each other.
2	startup_test_level_cap_db	-9.0	Caps the predicted RESPONSE trace here, not the drive trace, since H is in hand.
3	Ki	0.5	Loop gain on the Gauss-Newton step. 1.0 is deadbeat on the linearisation; 0 freezes the update. Nothing clamps it.
4	max_step_db	3.0	Maximum per-cycle change in any one channel's predicted RESPONSE. Bounded gains do not imply a bounded response move, so the limit is enforced on the response itself.
5	ridge	0.1	Tikhonov weight relative to $\text{tr}(S^{\wedge T} S)/N$, so it is scale-free.
6	resonance_sensitivity	0.5	Per-line gain scheduling from a self-scaling half-power peak detector. 0 disables.
7	ceiling_db	12.0	Anti-windup. The response may not exceed target by more than this, judged from the measured level. 0 disables.
—	projected_target_file	none	Any non-numeric field, at any position. Read on the first control cycle, not at construction. A stale projection is not detected.

pseudoinverse_control and buzz_control · control_laws.py:210

pos	name	default	meaning and extremes
0	rcond	1e-15	As match_trace.
1	max_drive_coherence	1.0	As match_trace.
2	startup_test_level_cap_db	-9.0	First command only.
3	running_ceiling_db	1e9 = OFF	Ceiling on every later command. DOCUMENTED AS +3.0 IN BOTH LAWS' DOCSTRINGS; the code default is off. Switch it on with a 4th value.

11. Corrections recorded in the code

These are kept because each one is a caution about a metric or an assumption, and because in several cases the wrong version had already been reported as a result.

- **The demotion gate was bitwise equality**, and the docstring's account of when Rattlesnake re-passes the FRF was wrong. Consequence: with the FRF update on, the fast law was byte-for-byte the base class, so any comparison of “fast” against “optimal diagonal” in that configuration was a law against itself.
- **Three statistics were taken over the whole FRF array** rather than in band: the singular-value spread (reference moved $0.0152 \rightarrow 0.1435$, a factor of 9), the gate's movement metric (median 4.37 against a true 0.17, a factor of 25), and the divergence guard (fired on ~1050 of 2049 bins every call while controlling perfectly). Each was written after diagnosing the previous one.
- **The startup cap compared incompatible units** and ran in the wrong order relative to the coherence cap. See §7.
- **drive_rcond was briefly gated to the dB error domain only**, leaving the factored path unconstrained in its default linear mode. Run 22 is what that cost: +14.29 dB measured against -0.39 predicted.
- **The rcond cache could be poisoned by a single bin**. A harness calling `_solve_one_bin` directly computed the singular-value reference from the one bin it was handed and cached it forever — found when an FRF sensitivity test reported 10.34 dB for a configuration the control loop scores at 5.95.
- **A constructor ordering bug** in the fast subclass was latent: state was set after `super().__init__()`, which can reach the overridden `_solve_one_bin`. It never fired in the app because Rattlesnake builds laws with no FRF, and fired immediately in a harness that supplies one.
- **“Run 13 had a weak system identification” was retracted**. The claim compared run 13's channel-7 minimum against other runs' non-channel-7 minima. Channel 7 sits between 0.28 and 0.54 in all thirteen identifications taken on this system — a property of the rig at the band edge. Scored like against like, run 13's identification was the best of the set. When judging one here, use the median and the 5th percentile, or per-channel minima compared like with like; a bare minimum over all channels and lines says nothing about run quality.
- **The “drive coherence goes to exactly 1.0” claim did not reproduce** at this scale. Established at 12 drives, it measured median 0.407 with 0.0 % of pairs at exactly 1.0 at 6 drives.
- **Synthetic benchmarks overstated the fast law's speedup** — 18–20× on random FRFs against about 3.6× on real data.

Documented defaults that are wrong in the docstrings

- `running_ceiling_db` is documented as “default +3.0” in both `pseudoinverse_control` and `buzz_control`. The code default is 1e9 — off.
- The fast law's docstring calls the dB error domain “the default since 2026-09-17”. The executable default is linear.
- The fast law's docstring lists seven parameter positions; the base class parses ten.
- Its `_restricted_basis` table still marks 1e-2 as the default; the shipping default is 2.0 *as a multiple*, about 3.0e-2 on this frame.
- `match_trace_pseudoinverse`'s docstring documents three parameters; the parser reads five.

Written 2026-09-20, revised 2026-09-21 (the loop classification corrected a second time, and the response-error trim added), against the source at branch `control-law-development-2026-09`, commit 9d2b522c. Flow charts from Claude outputs/flowcharts/*.mmd, regenerated to match this code. Companion document: *Control law comparison*, same date, which carries the measured results these mechanisms produced.